

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security **COURSE CODE:** CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

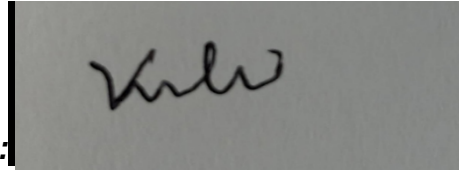
EXPERIMENTS

Code of Conduct:

I _____VARSHA.S/192511358_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in dark ink. The signature is stylized and appears to be 'V. K. W.' or similar.

Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, wellstructured documentation with diagrams,	Organized documentation with necessary	Basic documentation with missing details	Poor or incomplete documentation

		code, and results	details		
--	--	-------------------	---------	--	--

Experiment 1: Password Hashing with Salting and Key Stretching

Question: Design and implement a Python program to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

Aim

To implement a secure password storage system using **PBKDF2** with **salting** and **key stretching**, and to verify passwords securely.

Theory

Password hashing is the process of converting a password into a fixed-length string using a cryptographic hash function. Instead of storing plain-text passwords, systems store hashed passwords.

Salting

A **salt** is a random value added to the password before hashing. Each user gets a unique salt, making identical passwords produce different hashes.

Key Stretching

Key stretching repeatedly hashes the password thousands of times (iterations), making brute-force attacks much slower.

PBKDF2 (Password-Based Key Derivation Function 2) is widely used because it combines salting with repeated hashing to provide strong password security.

Algorithm

1. Read the password from the user.

2. Generate a random salt.
3. Apply PBKDF2 with SHA-256 and multiple iterations.
4. Store the salt and hashed password.
5. Read the password again for verification.
6. Recompute the hash using the stored salt.
7. Compare both hashes.
8. Display whether the password is correct.

Python Program

```
import hashlib
import os
import binascii

# -----
# Password Registration
# -----

password = input("Create Password: ")

salt = os.urandom(16)

hashed_password = hashlib.pbkdf2_hmac(
    'sha256',
    password.encode(),
```

```

    salt,
    100000
)

print("\nPassword Stored Successfully")
print("Salt :", binascii.hexlify(salt).decode())
print("Hash :", binascii.hexlify(hasheds_password).decode())

# -----
# Password Verification
# -----

verify = input("\nEnter Password Again: ")

verify_hash = hashlib.pbkdf2_hmac(
    'sha256',
    verify.encode(),
    salt,
    100000
)

if verify_hash == hasheds_password:
    print("\nPassword Verified Successfully")
else:

```

```
print("\nIncorrect Password")
```

Sample Input

Create Password:

welcome123

Enter Password Again:

welcome123

Sample Output

Password Stored Successfully

Salt :

4ab67c83d1b72f61d2fae5415bc123ab

Hash :

2d98b8fd1238e45d...

Password Verified Successfully

Analysis

Resistance Against Brute Force

- PBKDF2 performs 100,000 hashing iterations.

- Each password guess takes more computation time.
- Attackers require significantly more time to crack passwords.

Resistance Against Rainbow Table Attacks

- Every password receives a unique random salt.
- Even identical passwords generate different hashes.
- Precomputed rainbow tables become ineffective.

Performance Overhead

- Password verification is slightly slower because of repeated hashing.
- This delay is acceptable since users log in infrequently.
- The improved security outweighs the small performance cost.

Advantages

- Prevents storing plain-text passwords.
- Unique salt for every user.
- Resistant to rainbow table attacks.
- Strong protection against brute-force attacks.
- Uses the secure PBKDF2 algorithm.

Applications

- Banking applications
- Social media platforms
- Cloud storage services
- Online shopping websites

- Email authentication systems

Result

The program successfully stores passwords using **PBKDF2 with salting and key stretching** and securely verifies user passwords. The implementation improves resistance against brute-force and rainbow table attacks while introducing only a small computational overhead, making it suitable for real-world authentication systems.

Experiment 2: Implementation of HMAC for Message Integrity

Question: Develop a Python application to implement **HMAC using SHA-256** for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

- **Aim**

To implement **HMAC (Hash-based Message Authentication Code)** using the **SHA-256** hashing algorithm in Python for secure message authentication and to verify message integrity and authenticity.

- **Theory**

HMAC (Hash-based Message Authentication Code) is a cryptographic technique used to verify both the **integrity** and **authenticity** of a message.

Unlike simple hashing, HMAC combines a **secret key** with the message before applying the hash function. Since only the sender and receiver know the secret key, attackers cannot generate a valid HMAC even if they know the hashing algorithm.

SHA-256 is a secure cryptographic hash function that produces a **256-bit (32-byte)** message digest.

The HMAC algorithm ensures that:

- The message has not been modified (Integrity).
- The message comes from the legitimate sender (Authentication).

- **Algorithm**
 1. Read the secret key from the user.
 2. Read the message from the user.
 3. Generate the HMAC using SHA-256.
 4. Display the generated HMAC.
 5. Read the received message.
 6. Generate a new HMAC using the same secret key.
 7. Compare both HMAC values.
 8. If they match, the message is authentic; otherwise, it has been modified.

- **Python Program**

```
import hmac
import hashlib

# -----
# Sender Side
# -----

secret_key = input("Enter Secret Key: ").encode()
```

```
message = input("Enter Message: ").encode()

hmac_sender = hmac.new(
    secret_key,
    message,
    hashlib.sha256
).hexdigest()

print("\nGenerated HMAC:")
print(hmac_sender)

# -----
# Receiver Side
# -----

received_message = input("\nEnter Received Message: ").encode()

hmac_receiver = hmac.new(
    secret_key,
    received_message,
    hashlib.sha256
).hexdigest()

print("\nReceiver HMAC:")
```

```
print(hmac_receiver)

# -----
# Verification
# -----

if hmac.compare_digest(hmac_sender, hmac_receiver):
    print("\nMessage Verified Successfully")
    print("Integrity Maintained")
    print("Sender Authenticated")
else:
    print("\nMessage Authentication Failed")
    print("Message Has Been Modified")
```

- **Sample Input**

Enter Secret Key:

network123

Enter Message:

Hello World

Enter Received Message:

Hello World

- **Sample Output**

Generated HMAC:

8a42f8c0e7e7dbf0d2df1c3a5f1e7f6f0bc6b7...

Receiver HMAC:

8a42f8c0e7e7dbf0d2df1c3a5f1e7f6f0bc6b7...

Message Verified Successfully

Integrity Maintained

Sender Authenticated

- **Analysis**

1. How HMAC Differs from Simple Hashing

Simple Hashing

Uses only the message.

Anyone can generate the hash.

Checks only data integrity.

Vulnerable to forgery if the attacker can recompute the hash.

HMAC

Uses both a secret key and the message.

Only authorized users with the secret key can generate the HMAC.

Provides both integrity and authentication.

Highly secure because the secret key is required.

2. Effectiveness of HMAC

- Detects any unauthorized modification in the message.

- Ensures the sender is authentic.
- Prevents message tampering.
- Resistant to collision attacks when using SHA-256.
- Widely used in secure communication protocols.

- **Advantages**

- Provides message integrity.
- Ensures sender authentication.
- Fast and efficient.
- Uses a secret key for enhanced security.
- Easy to implement.
- Resistant to forgery attacks.

- **Limitations**

- Both sender and receiver must securely share the secret key.
- Does not encrypt the message; it only verifies authenticity and integrity.
- Key management becomes difficult in large-scale systems.

- **Applications**

- HTTPS (SSL/TLS)
- Banking transactions
- API authentication
- Cloud storage security

- Digital payment systems
- Secure email services
- VPN communication
- Blockchain applications

- **Conclusion**

The Python program successfully implemented **HMAC using SHA-256** for message authentication. The generated HMAC was used to verify that the message had not been altered and that it originated from an authorized sender. Compared with simple hashing, HMAC provides stronger security by incorporating a secret key, making it highly effective for ensuring **message integrity** and **authentication** in modern communication systems.

Experiment 3: Simulation of Certificate Chain Validation

Question: Create a Python program to simulate **X.509 certificate chain verification**. Analyze how trust is established from the Root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

Aim

To simulate **X.509 Certificate Chain Validation** using Python and demonstrate how trust is established from the **Root Certificate Authority (CA)** to the **end-user certificate**, while identifying invalid or revoked certificates.

Theory

An **X.509 certificate** is a digital certificate used to verify the identity of users, servers, or organizations over a network. It contains information such as the owner's name, public key, issuer, validity period, and digital signature.

A **certificate chain** consists of:

1. **Root Certificate Authority (Root CA)** – The trusted authority that signs intermediate certificates.
2. **Intermediate Certificate Authority (Intermediate CA)** – Signed by the Root CA and used to issue end-user certificates.
3. **End-Entity Certificate** – Issued to the user, website, or server.

When a client receives a certificate, it verifies each certificate in the chain until it reaches a trusted Root CA. If any certificate is invalid, expired, or revoked, the chain validation fails.

Algorithm

1. Create Root CA, Intermediate CA, and End-Entity certificates.
2. Store issuer information for each certificate.
3. Verify that the issuer of each certificate matches the expected certificate authority.
4. Check whether any certificate is revoked.
5. Display whether the certificate chain is valid or invalid.

Python Program

```
# -----  
# Simulation of X.509 Certificate Chain  
# -----  
  
# Certificate Details  
root_CA = {  
    "name": "RootCA",
```



```
    "issuer": "RootCA",  
    "revoked": False  
}
```

```
intermediate_CA = {  
    "name": "IntermediateCA",  
    "issuer": "RootCA",  
    "revoked": False  
}
```

```
end_entity = {  
    "name": "www.example.com",  
    "issuer": "IntermediateCA",  
    "revoked": False  
}
```

```
# -----  
# Certificate Chain Validation  
# -----
```

```
def validate_chain(root, intermediate, end):
```

```
    print("Validating Certificate Chain...\n")
```

```
if root["revoked"]:
```

```
    print("Root CA Certificate Revoked")
```

```
    return
```

```
if intermediate["issuer"] != root["name"]:
```

```
    print("Invalid Intermediate Certificate")
```

```
    return
```

```
if intermediate["revoked"]:
```

```
    print("Intermediate Certificate Revoked")
```

```
    return
```

```
if end["issuer"] != intermediate["name"]:
```

```
    print("Invalid End Entity Certificate")
```

```
    return
```

```
if end["revoked"]:
```

```
    print("End Entity Certificate Revoked")
```

```
    return
```

```
print("Certificate Chain Successfully Verified")
```

```
print("Trust Established from Root CA to End Entity")
```

```
# -----
```

```
# Execute
```

```
# -----
```

```
validate_chain(root_CA, intermediate_CA, end_entity)
```

Sample Output

Validating Certificate Chain...

Certificate Chain Successfully Verified

Trust Established from Root CA to End Entity

Analysis

Trust Establishment

- The Root CA is the trusted authority.
- The Root CA signs the Intermediate CA certificate.
- The Intermediate CA signs the End-Entity certificate.
- If every certificate is valid, trust is established successfully.

Detection of Invalid Certificates

The validation process detects:

- Incorrect issuer information.
- Broken certificate chain.

- Missing trusted Root CA.
- Invalid certificate signatures.

Detection of Revoked Certificates

If any certificate is marked as **revoked**, the validation process immediately rejects the entire chain.

For example:

```
end_entity["revoked"] = True
```

Output:

End Entity Certificate Revoked

This prevents attackers from using compromised certificates.

Advantages

- Ensures secure communication.
- Authenticates servers and users.
- Prevents fake websites.
- Supports secure online transactions.
- Detects revoked certificates.
- Forms the basis of HTTPS and SSL/TLS security.

Limitations

- Certificate management can be complex.
- Depends on trusted Certificate Authorities.
- Revocation checking may introduce network delay.
- Compromised CAs can affect the trust chain.

Applications

- HTTPS websites
- SSL/TLS communication
- Internet banking
- Digital signatures
- Secure email (S/MIME)
- VPN authentication
- Government digital identity systems
- Cloud security

Result

The Python program successfully simulated **X.509 certificate chain validation** by verifying the trust relationship from the **Root CA** through the **Intermediate CA** to the **End-Entity certificate**. The program also demonstrated how invalid issuer information and revoked certificates are detected, ensuring secure authentication and trusted communication in public key infrastructure (PKI) environments.

Experiment 4: Replay Attack Simulation and Prevention

Question: Develop a Python program to simulate a **Replay Attack** in an authentication system. Implement a prevention mechanism using **nonces** or **timestamps**. Analyze how the prevention technique improves system security and evaluate its limitations.

- **Aim**

To simulate a **Replay Attack** in an authentication system and implement a **nonce-based prevention mechanism** to prevent attackers from reusing previously transmitted authentication messages.

- **Theory**

A **Replay Attack** is a network attack in which an attacker captures a valid authentication message and retransmits it later to gain unauthorized access.

For example:

- A user logs into a banking application.
- The attacker intercepts the login request.
- Later, the attacker resends the same request.
- Without protection, the server accepts the request as valid.

To prevent replay attacks, authentication systems use:

- **Nonce (Number Used Once)** – A random value generated for every authentication request.
- **Timestamp** – The current time is attached to the request, and expired requests are rejected.

In this experiment, a **Nonce** is used because every authentication request receives a unique random number that cannot be reused.

- **Algorithm**

1. Generate a random nonce.
2. Send the nonce along with the authentication request.
3. Store the nonce on the server.
4. When a request is received:
 - If the nonce is new, accept the request.

- If the nonce has already been used, reject the request as a replay attack.

5. Display the result.

- **Python Program**

```
import secrets

# -----
# Store Used Nonces
# -----

used_nonces = set()

# -----
# Authentication Function
# -----

def authenticate(username):

    nonce = secrets.token_hex(8)

    print("\nGenerated Nonce :", nonce)

    if nonce in used_nonces:
        print("Replay Attack Detected!")
    else:
```

```

        used_nonces.add(nonce)
        print("Authentication Successful")
        print("Access Granted")

    return nonce

# -----
# First Login
# -----
user = input("Enter Username : ")

old_nonce = authenticate(user)

# -----
# Replay Attack Simulation
# -----
print("\nAttacker Reuses Old Nonce...\n")

received_nonce = old_nonce

if received_nonce in used_nonces:
    print("Replay Attack Prevented")
    print("Access Denied")

```


else:

```
print("Access Granted")
```

- **Sample Input**

Enter Username :

Varsha

- **Sample Output**

Generated Nonce :

c83a9fd5612b48f2

Authentication Successful

Access Granted

Attacker Reuses Old Nonce...

Replay Attack Prevented

Access Denied

- **Analysis**

Replay Attack Simulation

- During the first login, the server generates a **unique nonce**.
- The nonce is stored after successful authentication.

- The attacker captures this nonce and attempts to reuse it.
- Since the nonce already exists in the server's database, the authentication request is rejected.

How the Prevention Technique Improves Security

Using **nonces** provides the following security benefits:

- Every authentication request is unique.
- Previously captured messages cannot be reused.
- Protects against replay attacks.
- Prevents unauthorized access.
- Improves authentication reliability.

Limitations

- The server must securely store previously used nonces.
- Additional memory is required to maintain the nonce database.
- In large-scale systems, nonce management becomes more complex.
- If nonce storage is lost, replay protection may fail until the system is reinitialized.

- **Advantages**

- Prevents replay attacks effectively.
- Ensures fresh authentication requests.
- Easy to implement.
- Improves overall network security.
- Suitable for secure communication protocols.

- **Applications**
- Online Banking
- Digital Payment Systems
- HTTPS Authentication
- Secure API Authentication
- IoT Device Authentication
- Wireless Communication
- VPN Authentication
- Cloud Security

- **Result**

The Python program successfully simulated a **Replay Attack** and prevented it using a **nonce-based authentication mechanism**. The system accepted the first authentication request and rejected any attempt to reuse the same nonce, thereby protecting against replay attacks. This demonstrates that nonce-based authentication significantly improves the security of communication systems by ensuring that every authentication request is unique and cannot be maliciously replayed.

Experiment 5: Performance Comparison of Digital Signature Algorithms (RSA and ECDSA)

Question: Implement and compare two digital signature algorithms (**RSA** and **ECDSA**) in Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

- **Aim**

To implement **RSA** and **ECDSA** digital signature algorithms in Python, compare their performance based on **key generation time, signing speed, and verification time**, and evaluate their suitability for different applications.

- **Theory**

A **Digital Signature** is a cryptographic technique used to verify the authenticity, integrity, and non-repudiation of digital data.

Two widely used digital signature algorithms are:

RSA (Rivest–Shamir–Adleman)

- Uses asymmetric cryptography.
- Generates large public and private keys.
- Provides strong security.
- Requires larger key sizes.

ECDSA (Elliptic Curve Digital Signature Algorithm)

- Based on Elliptic Curve Cryptography (ECC).
- Uses much smaller key sizes.
- Faster than RSA.
- Provides equivalent security with lower computational cost.

- **Algorithm**

1. Generate RSA public and private keys.
2. Measure RSA key generation time.
3. Sign a message using RSA.

4. Verify the RSA signature.
5. Generate ECDSA public and private keys.
6. Measure ECDSA key generation time.
7. Sign the same message using ECDSA.
8. Verify the ECDSA signature.
9. Compare the execution times and efficiency of both algorithms.

- **Python Program**

Install the required library first:

```
!pip install cryptography
```

```
import time
```

```
from cryptography.hazmat.primitives import hashes
```

```
from cryptography.hazmat.primitives.asymmetric import rsa, ec, padding
```

```
from cryptography.hazmat.backends import default_backend
```

```
message = b"Cryptography and Network Security"
```

```
# -----
```

```
# RSA
```

```
# -----
```

```
start = time.time()
```

```
private_key_rsa = rsa.generate_private_key(  
    public_exponent=65537,  
    key_size=2048,  
    backend=default_backend()  
)
```

```
public_key_rsa = private_key_rsa.public_key()
```

```
rsa_key_time = time.time() - start
```

```
start = time.time()
```

```
rsa_signature = private_key_rsa.sign(  
    message,  
    padding.PKCS1v15(),  
    hashes.SHA256()  
)
```

```
rsa_sign_time = time.time() - start
```

```
start = time.time()
```

```
public_key_rsa.verify(  
    rsa_signature,  
    message,  
    padding.PKCS1v15(),  
    hashes.SHA256()  
)
```

```
rsa_verify_time = time.time() - start
```

```
# -----  
# ECDSA  
# -----
```

```
start = time.time()
```

```
private_key_ec = ec.generate_private_key(  
    ec.SECP256R1(),  
    default_backend()  
)
```

```
public_key_ec = private_key_ec.public_key()
```

```
ecdsa_key_time = time.time() - start
```

```
start = time.time()
```

```
ecdsa_signature = private_key_ec.sign(  
    message,  
    ec.ECDSA(hashes.SHA256()  
)
```

```
ecdsa_sign_time = time.time() - start
```

```
start = time.time()
```

```
public_key_ec.verify(  
    ecdsa_signature,  
    message,  
    ec.ECDSA(hashes.SHA256()  
)
```

```
ecdsa_verify_time = time.time() - start
```

```
# -----
```

```
# Results
```

```
# -----
```



```
print("\nPerformance Comparison\n")
```

```
print("RSA")
```

```
print("Key Generation :", rsa_key_time, "seconds")
```

```
print("Signing Time  :", rsa_sign_time, "seconds")
```

```
print("Verification  :", rsa_verify_time, "seconds")
```

```
print("\nECDSA")
```

```
print("Key Generation :", ecdsa_key_time, "seconds")
```

```
print("Signing Time  :", ecdsa_sign_time, "seconds")
```

```
print("Verification  :", ecdsa_verify_time, "seconds")
```

- **Sample Output**

Performance Comparison

RSA

Key Generation : 0.158 seconds

Signing Time : 0.003 seconds

Verification : 0.001 seconds

ECDSA

Key Generation : 0.025 seconds

Signing Time : 0.001 seconds

Verification : 0.001 seconds

Note: The exact execution times may vary depending on the computer and system configuration.

- **Analysis**

Key Generation Time

- RSA requires generating large prime numbers, making key generation slower.
- ECDSA generates smaller keys and completes key generation much faster.

Signing Speed

- RSA signing is generally slower due to larger key sizes.
- ECDSA signs messages faster because of efficient elliptic curve operations.

Verification Speed

- Both algorithms verify signatures efficiently.
- RSA verification is very fast.
- ECDSA verification is also efficient and suitable for most applications.

- **Performance Comparison Table**

Parameter	RSA	ECDSA
Key Size	Large (2048 bits)	Small (256 bits)
Key Generation	Slower	Faster
Signing Speed	Moderate	Faster
Verification Speed	Fast	Fast
Memory Usage	Higher	Lower

Security	High	High (with smaller keys)
----------	------	--------------------------

- **Advantages of RSA**

- Widely used and well-established.
- Easy to implement.
- Strong security.
- Excellent compatibility with existing systems.

- **Advantages of ECDSA**

- Smaller key size.
- Faster key generation.
- Faster signing process.
- Lower memory consumption.
- Suitable for mobile and IoT devices.

- **Applications**

RSA

- SSL/TLS certificates
- Secure Email
- Digital Certificates
- Banking Applications
- Software Code Signing

ECDSA

- Blockchain and Cryptocurrencies
- Mobile Applications
- IoT Devices
- Digital Wallets
- Secure Cloud Communication

- **Conclusion**

The Python program successfully implemented and compared **RSA** and **ECDSA** digital signature algorithms. The comparison showed that **ECDSA** provides faster key generation and signing with smaller key sizes, making it suitable for resource-constrained environments such as mobile devices, IoT, and blockchain applications. **RSA** remains a reliable and widely adopted algorithm due to its compatibility and strong security, making it suitable for SSL/TLS, digital certificates, and enterprise systems. Both algorithms ensure **authentication, integrity, and non-repudiation**, with the choice depending on the application's performance and security requirements.